1. Docker (30 minutes)

Goal

Containerize the hello-world Python app.

Steps

- Create a Dockerfile to generate the Docker image

Requirements

- The webapp should be served by **passenger + nginx**
- Nginx access logs should be redirected to stdout
- Nginx error logs should be redirected to stderr

Files to submit

- The Dockerfile and its attached files

2. Terraform (30 minutes)

Goal

Create a small GKE(Google Cloud) or EKS(AWS) cluster **with Terraform**.

Steps

- Write a Terraform module to deploy the cluster
- Apply it to your cloud project

Requirements

- Cluster
  - Zonal, or single AZ for AWS
  - Minimal Kubernetes version should be 1.22
- 1 node pool attached to the cluster
  - Initial node count is 1
  - Autoscaling enabled from 1 to 3 instances
  - Node type is preemptible/spot instance (cheap instances)

Files to submit

- The Terraform module

3. Kubernetes (30 minutes)

Goal

Deploy the hello-world app into the GKE/AWS cluster created.

Steps

- Write Kubernetes manifests/templates/code
- Apply them to your cluster

Requirements

- App should have a load balancer

Bonus Points

- The app is accessible using HTTPS. You can use letsencrypt-staging to create the certificates or another certificate provider.
- The app is able to autoscale depending on CPU usage from 2 to 4 instances
- The app has an healthcheck setup with appropriate parameters
- Updating the app should not have downtime.
- The app must be protected from full eviction (all instances removed at the same time)
- Document your solution (architecture, deployment) as if you are explaining it to a new engineer with no background in Kubernetes (this is a test of your communication skills)

Files to submit

- The Kubernetes manifests, templates, scripts wrote to deploy the app
- Your solution’s documentation